

Programación de microcontroladores

Interrupciones

Juan José Londoño, M.Sc.

Introducción

Hasta ahora se ha trabajado con *polling* en las entradas, es decir, la CPU revisa constantemente si ocurrió algún evento.

Este método funciona, pero tiene un problema importante, el procesador pierde tiempo revisando eventos que tal vez no han ocurrido.

En sistemas más complejos se necesita que el microcontrolador **solo reaccione cuando algo realmente sucede**.

¿Qué es una interrupción?

Es un mecanismo que permite al microcontrolador detener temporalmente la ejecución normal del programa para atender un evento importante.

Cuando se configura una interrupción, el microcontrolador:

1. Pausa el programa actual.
2. Ejecuta una función especial llamada rutina de servicio de interrupción (**ISR**).
3. Atiende el evento.
4. Regresa al programa donde se quedó.

Eventos que generan una interrupción

Un microcontrolador puede generar interrupciones por muchos eventos
Presionar un botón.

- Finalización de un timer.
- Recepción de datos por UART.
- Conversión terminada en ADC.
- Cambio en un pin digital.
- Comunicación SPI / I2C.

Tipos de interrupciones

Interrupciones externas

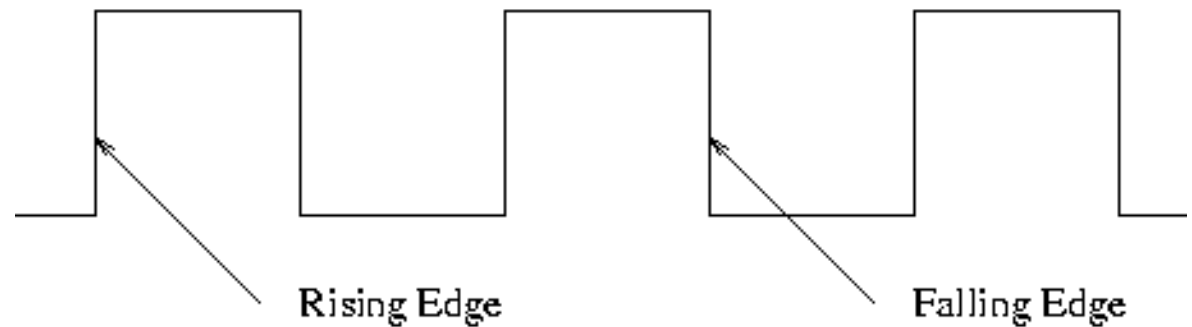
Proviene de pines del microcontrolador (presionar un botón o señal digital externa de otro circuito).

Se activan por:

Flanco de subida (***Rising edge***).

Flanco de bajada (***Falling edge***).

Cambio de estado.



Tipos de interrupciones

Interrupciones internas

Proviene de periféricos internos del microcontrolador, como:

- Timer.
- UART.
- ADC.
- SPI.
- I2C.

Consideraciones de las ISR

Las rutinas de servicio de interrupción (ISR) deben ejecutarse **lo más rápido posible** y contener la menor cantidad de código posible.

Esto se debe a que mientras una interrupción se está ejecutando:

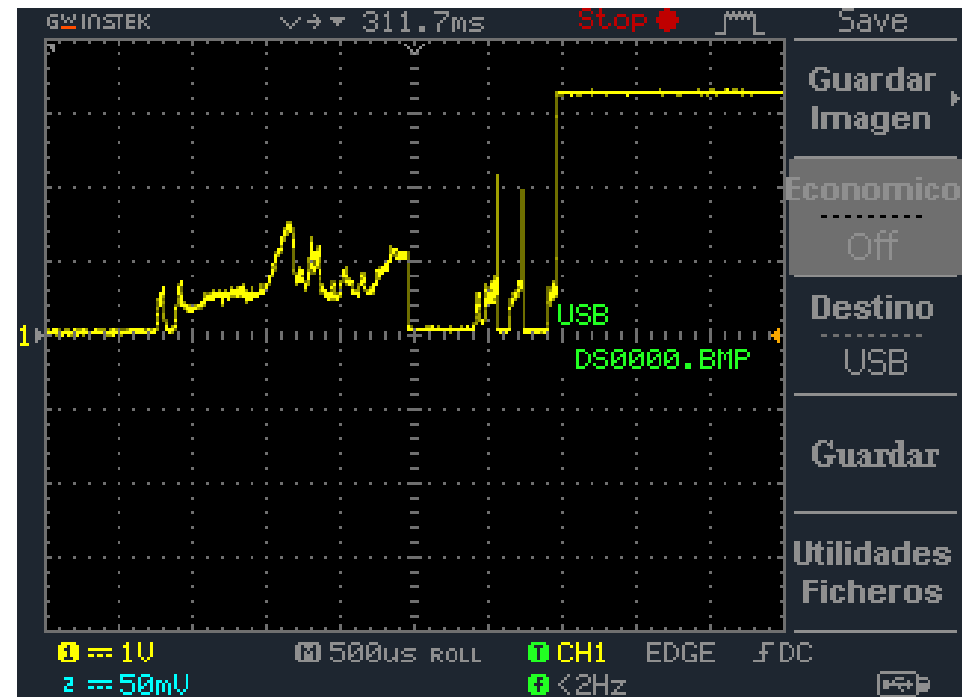
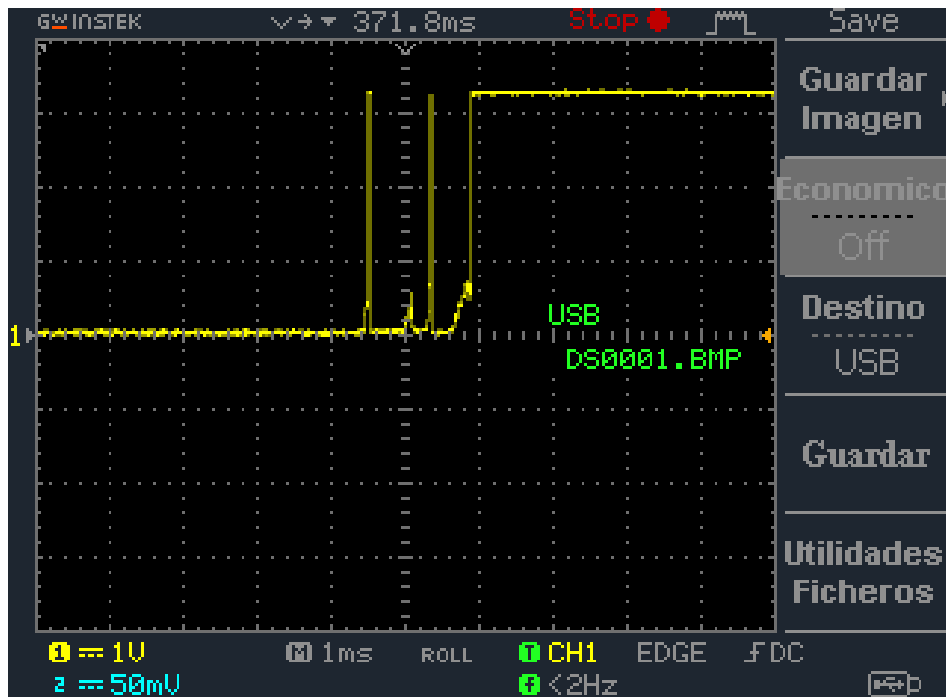
- El programa principal queda detenido.
- Otras interrupciones pueden retrasarse o perderse.
- El sistema puede volverse menos eficiente.

JAMÁS USAR DELAYS EN ISR



Consideraciones de las ISR

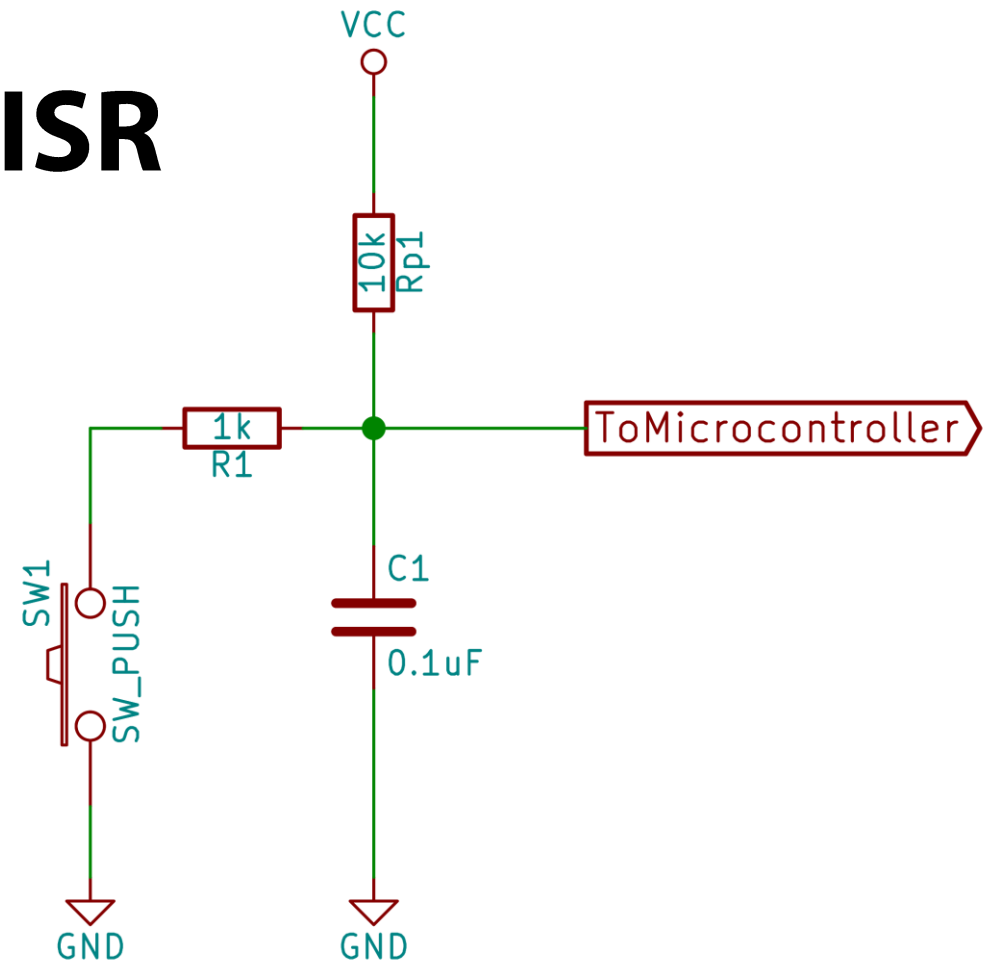
Rebotes

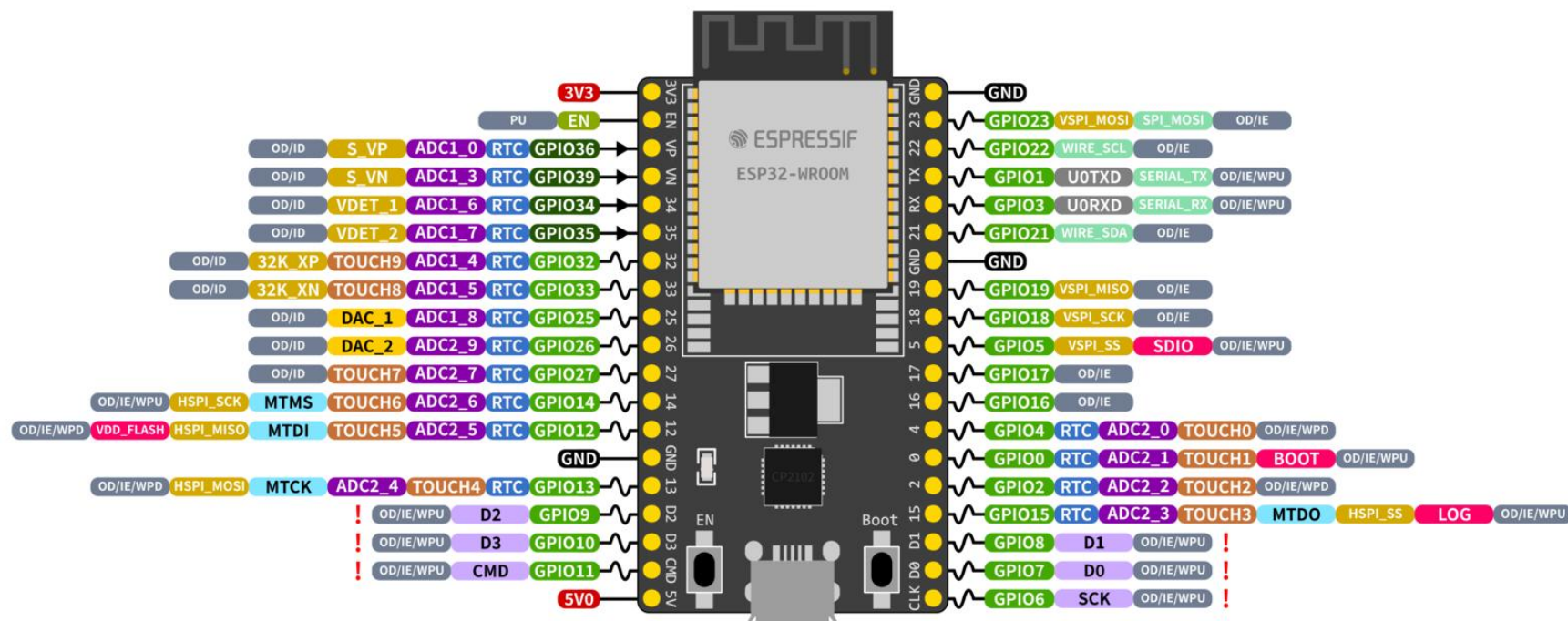


Consideraciones de las ISR

Rebotes

En las ISR no deben usarse *delays* ni funciones bloqueantes, pero se debe implementar una medida para atenuar los rebotes de los pulsadores que disparan una ISR.





Habilitar interrupción en GPIO

```
gpio_config_t button_config = {  
    .pin_bit_mask = (1ULL << BUTTON_GPIO),  
    .mode = GPIO_MODE_INPUT,  
    .pull_up_en = GPIO_PULLUP_ENABLE,  
    .pull_down_en = GPIO_PULLDOWN_DISABLE,  
    .intr_type = GPIO_INTR_NEGEDGE // Flanco de bajada  
};  
  
gpio_config(&button_config);
```

Instalar servicio global de interrupciones

```
gpio_install_isr_service(0);  
gpio_isr_handler_add(BUTTON_GPIO, button_isr_handler, NULL);
```

Variables bandera en ISR

Como las ISR no deben contener funciones pesadas, generalmente se usan para habilitar una bandera que avise al código principal la ocurrencia de un evento para que este ejecute la tarea pesada.

Esta variable debe ser declarada como ***volatile*** para que el compilador sepa que puede cambiar por fuera del flujo normal del programa.



```
static volatile bool led_state = false;
```

Declaración de ISR

```
static void IRAM_ATTR button_isr_handler(void *arg) {  
    // Código de la ISR  
}
```

ISR con Timers

Se puede configurar una alarma en un Timer para que cuando llegue a cierto valor, dispare una ISR. Esto permite automatizar eventos y evitar el *polling*.

Esta interrupción permite que se manejen eventos de forma inmediata cuando se cumple un tiempo determinado.

Configuración de ISR con Timers

```
timer_config_t config = {  
    .divider = TIMER_DIVIDER,  
    .counter_dir = TIMER_COUNT_UP,  
    .counter_en = TIMER_PAUSE,  
    .alarm_en = TIMER_ALARM_EN,  
    .auto_reload = true  
};  
  
timer_init(TIMER_GROUP_0, TIMER_0, &config);  
  
timer_set_counter_value(TIMER_GROUP_0, TIMER_0, 0);
```

Configuración de ISR con Timers

```
timer_set_alarm_value(  
    TIMER_GROUP_0,  
    TIMER_0,  
    TIMER_INTERVAL // Ticks en los que se dispara la ISR  
);  
  
timer_isr_callback_add(  
    TIMER_GROUP_0,  
    TIMER_0,  
    timer_isr_callback, // ISR que se desea registrar  
    NULL,  
    0  
);
```

Configuración de ISR con Timers



```
timer_enable_intr(TIMER_GROUP_0, TIMER_0);  
timer_start(TIMER_GROUP_0, TIMER_0);
```

Declaración de ISR con Timers

A diferencia de las ISR para interrupciones externas, las ISR para alarmas de los timers en el ESP32 se deben declarar con un tipo de dato de retorno ***booleano*** y deben retornar ***false***.

```
static bool IRAM_ATTR timer_isr(void *args) {  
    // Código de la ISR  
    return false;  
}
```